

8



Case Study: Hotel Pricing System

In this chapter, we offer a case study in which ADD is used to design a greenfield system in a mature domain. This case study presents the early design rounds, which include four iterations, and is based on a real-world example. We first describe the business context for the system, then summarize its requirements. This is followed by a step-by-step summary of the activities that are performed during the ADD iterations.

Why Read This Chapter?

People learn best from examples. This chapter will help you to contextualize and visualize how the many concepts that have been previously introduced can be put together in an actual setting. We do this with a detailed example of addressing a real-world problem—a hotel pricing system—with an architectural solution.

8.1 Business Case

AD&D Hotels is a mid-sized business hotel chain (currently around 300 hotels) that has been experiencing robust growth in recent years. The IT infrastructure of the company is composed of many different applications, such as a Property Management System, a Commercial Analysis System, an Enterprise Reservation System, and a Channel Management System. At the center of this system of systems is the Hotel Pricing System, as shown in the context diagram in Figure 8.1. Although this system was already deployed in the cloud, the approach followed was “lift and shift”; thus, this system was not making full use of cloud resources.

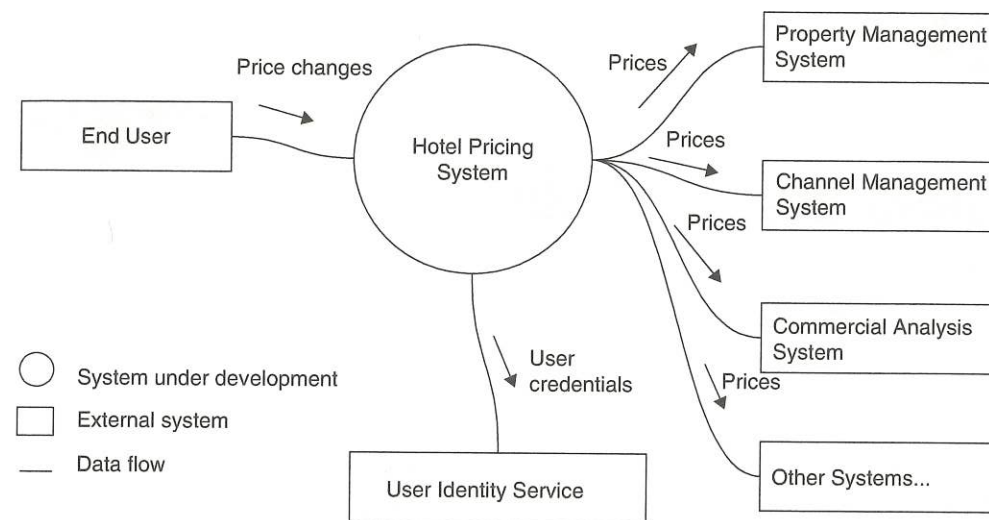


FIGURE 8.1 Context diagram for the Hotel Pricing System

The Hotel Pricing System (HPS) is used by sales managers and commercial representatives to establish prices for rooms on specific dates for the different hotels in the company. Prices are associated with different rates (e.g., a public rate, a discount rate), and most of the prices for the different rates are calculated by taking a base rate and applying business rules to it (although some of the rates can also be fixed and not depend on the base rate). Managers typically change prices of the base and fixed rates. Then, using this information, the HPS calculates the prices of all the rates for all rooms in all hotels, which also vary according to the types of rooms available in each hotel. Prices that are calculated by the HPS are used by other systems in the company to make reservations, and are also sent to different online travel agencies through the Channel Management System (CMS). The company's systems are hosted with a cloud provider that offers a user identity service that manages users and provides single sign-on functionalities.

AD&D Hotels wants to modernize its IT infrastructure. The first step will be the complete replacement of the existing pricing system, which was developed several years ago and is suffering from reliability, performance, availability, and maintainability issues that have resulted in financial losses. Furthermore, the company has experienced difficulties because many of its systems are connected using traditional SOAP and REST request-response endpoints: Changes to one application frequently impact other applications and complicate the deployment of individual updates to specific applications. Also, the failure of an application can propagate through the entire system. Furthermore, some of the applications interact

using what are now well-known anti-patterns, such as integration through a shared database. Recently revised enterprise architecture principles within the organization mandate a migration of the system toward a more decoupled model. The remainder of this chapter describes the first three iterations of an architectural design for this system, created using ADD.

8.2 System Requirements

Requirement elicitation activities were performed earlier by the company. This section summarizes the most important requirements collected.

8.2.1 Primary Functionality

The HPS's functionality is conceptually simple. The main user stories for this system are shown in the use case diagram in Figure 8.2.

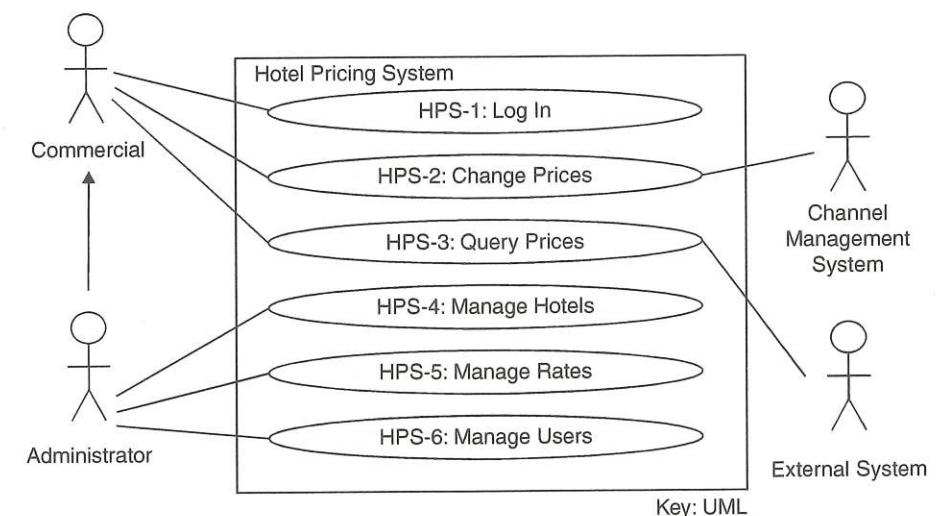


FIGURE 8.2 Initial use case diagram for the Hotel Pricing System

Each of these user cases is described in Table 8.1.

TABLE 8.1 Use Cases for the Hotel Pricing System

Use Case	Description
HPS-1: Log In	A user (commercial or administrator) provides their credentials in a login window. The system checks these credentials against a user identity service and, if successful, provides access to the system. Once logged in, a user can only make queries and changes to the hotels for which they have been authorized.
HPS-2: Change Prices	A user selects a specific hotel for which they are authorized to change prices, and selects dates where they want to make price changes to either a base rate or a fixed rate. All of the prices for the rates that are calculated from the base rate are calculated at that point. The system allows price changes to be simulated before they are actually changed. When the prices are changed, they are pushed to the Channel Management System and become available for querying by external systems.
HPS-3: Query Prices	A user or an external system queries prices for a given hotel through the user interface or a query API.
HPS-4: Manage Hotels	An administrator adds, changes, or modifies hotel information. This includes editing the hotel's tax rates, available rates, and room types.
HPS-5: Manage Rates	An administrator adds, changes, or modifies rates. This includes defining the calculation business rules for the different rates.
HPS-6: Manage Users	An administrator changes permissions for a given user.

8.2.2 Quality Attribute Scenarios

In addition to these use cases, a number of quality attribute scenarios were elicited and documented. The seven most relevant and important ones are presented in Table 8.2. For each scenario, we also identify the use case that it is associated with.

TABLE 8.2 Quality Attribute Scenarios for the Hotel Pricing System

ID	Quality Attribute	Scenario	Associated Use Case
QA-1	Performance	A base rate price is changed for a specific hotel and date during normal operation; the prices for all the rates and room types for the hotel are published (ready for query) in less than 100 ms.	HPS-2
QA-2	Reliability	A user performs multiple price changes on a given hotel; 100% of the price changes are published (available for query) successfully and are also received by the Channel Management System.	HPS-2
QA-3	Availability	Pricing queries uptime SLA must be 99.9% outside of maintenance windows.	All
QA-4	Scalability	The system will initially support a minimum of 100,000 price queries per day through its API and should be capable of handling up to 1,000,000 without decreasing average latency by more than 20%.	HPS-3

ID	Quality Attribute	Scenario	Associated Use Case
QA-5	Security	A user logs into the system through the front-end. The credentials of the user are validated against the User Identity Service and, once logged in, they are presented with only the functions that they are authorized to use.	All
QA-6	Modifiability	Support for a price query endpoint with a different protocol than REST (e.g., gRPC) is added to the system. The new endpoint does not require changes to be made to the core components of the system.	All
QA-9	Testability	100% of the system and its elements should support integration testing independently of the external systems.	All

8.2.3 Constraints

Finally, a set of constraints on the system and its implementation were collected. These are presented in Table 8.3.

TABLE 8.3 Constraints for the Hotel Pricing System

ID	Constraint
CON-1	Users must interact with the system through a web browser in different platforms (Windows, OSX, and Linux, and different devices).
CON-4	The initial release of the system must be delivered in 6 months, but an initial version of the system (MVP) must be demonstrated to internal stakeholders in at most 2 months.
CON-5	The system must interact initially with existing systems through REST APIs but may need to later support other protocols.

8.2.4 Architectural Concerns

Since this is a greenfield development, only a few general concerns were identified initially and these are shown in Table 8.4.

TABLE 8.4 Architectural Concerns for the Hotel Pricing System

ID	Concern
CRN-1	Establish an overall initial system structure.
CRN-2	Leverage the team's knowledge about Java technologies, the Angular framework, and Kafka.
CRN-3	Allocate work to members of the development team.
CRN-4	Avoid introducing technical debt (see Chapter 10).

8.3 Development and Operations Requirements

As part of the modernization effort, AD&D Hotels wants to integrate agile (specifically Scrum) and DevOps practices into the development of the HPS. Besides the previously discussed system requirements, other development and operational requirements need to be taken into account.

Artifacts in the development process move through four different environments, as depicted in Figure 8.3. The different environments are the following:



FIGURE 8.3 Different environments during the development process

- *Development:* This is a local environment on the developers' computers.
- *Integration:* This is an environment in the cloud where an integrated version of the HPS is tested. In this environment, the system is not connected with all of the external systems, so some of these external systems are substituted by mock-ups.
- *Staging:* This is an environment in the cloud where the system's final tests (including load testing) are performed prior to deployment. Here the system is connected to test versions of all the external systems. At the end of a sprint, the system is typically demonstrated from this environment.
- *Production:* This is the real-world execution environment.

The company is just starting to implement sophisticated deployability practices, so only support for continuous deployment to the integration and staging environments is currently required. These new requirements, specific to development and operations, result in additional constraints, quality attribute scenarios, and concerns, which are presented next.

8.3.1 Quality Attribute Scenarios

In addition to the quality attribute scenarios that are part of the initial set of system requirements, several scenarios result from development and operations considerations. These scenarios are captured in Table 8.5.

TABLE 8.5 Development and Operations Quality Attribute Scenarios for the Hotel Pricing System

ID	Quality Attribute	Scenario	Associated Use Case
QA-7	Deployability	The application is moved between nonproduction environments as part of the development process. No changes in the code are needed.	All
QA-8	Monitorability	A system operator wishes to measure the performance and reliability of price publication during operation. The system provides a mechanism that allows 100% of these measures to be collected as needed.	HPS-2

8.3.2 Constraints

The additional constraints on the system and its implementation due to considerations of development and operations were collected. These are presented in Table 8.6.

TABLE 8.6 Development and Operations Constraints for the Hotel Pricing System

ID	Constraint
CON-2	Manage users through cloud provider identity service and host resources in the cloud.
CON-3	Code must be hosted on a proprietary Git-based platform that is already in use by other projects in the company.
CON-6	A cloud-native approach should be favored when designing the system.

8.3.3 Architectural Concerns

Development and operations requirements introduce a new concern, shown in Table 8.7.

TABLE 8.7 Development and Operations Architectural Concerns for the Hotel Pricing System

ID	Concern
CRN-5	Set up a continuous deployment infrastructure.

Given these sets of inputs, we are now ready to describe the design process, following the steps outlined in Chapter 4. In this chapter, we present only the final results of the requirements collection process. The job of collecting these requirements is nontrivial, but beyond the scope of this chapter.

8.4 The Software Design Process

We are now ready to make the leap from the world of requirements and business concerns to the world of design. This is perhaps the most important job for an architect—translating requirements into design decisions. Of course, many other decisions and duties are important, but this is the core of what it means to be an architect: making design decisions with far-reaching consequences.

8.4.1 ADD Step 1. Review Inputs

The first step of the ADD method involves reviewing the inputs and identifying which requirements will be considered as architectural drivers (which will be included in the design backlog). The inputs are summarized in Table 8.8.

TABLE 8.8 Inputs for the Hotel Pricing System

Category	Details																														
Design purpose	This project can be considered greenfield development, as it involves the complete replacement of an existing system. The purpose of the design activity is to make initial decisions to support the construction of the system from scratch.																														
Primary functional requirements	From the user stories presented in Section 8.2.1, the primary ones were determined to be: ▪ HPS-2: Change Prices—Because it directly supports the core business ▪ HPS-3: Query Prices—Because it directly supports the core business ▪ HPS-4: Manage Hotels—Because it establishes a basis for many other user stories																														
Quality attribute scenarios	The scenarios for the HPS have now been prioritized (as discussed in Section 2.4.2) as follows: <table><tr><th>Scenario ID</th><th>Importance to the Customer</th><th>Difficulty of Implementation According to the Architect</th></tr><tr><td>QA-1: Performance</td><td>High</td><td>High</td></tr><tr><td>QA-2: Reliability</td><td>High</td><td>High</td></tr><tr><td>QA-3: Availability</td><td>High</td><td>High</td></tr><tr><td>QA-4: Scalability</td><td>High</td><td>High</td></tr><tr><td>QA-5: Security</td><td>High</td><td>Medium</td></tr><tr><td>QA-6: Modifiability</td><td>Medium</td><td>Medium</td></tr><tr><td>QA-7: Deployability</td><td>Medium</td><td>Medium</td></tr><tr><td>QA-8: Monitorability</td><td>Medium</td><td>Medium</td></tr><tr><td>QA-9: Testability</td><td>Medium</td><td>Medium</td></tr></table> From this list, QA-1, QA-2, QA-3, QA-4, and QA-5 are selected as primary drivers.	Scenario ID	Importance to the Customer	Difficulty of Implementation According to the Architect	QA-1: Performance	High	High	QA-2: Reliability	High	High	QA-3: Availability	High	High	QA-4: Scalability	High	High	QA-5: Security	High	Medium	QA-6: Modifiability	Medium	Medium	QA-7: Deployability	Medium	Medium	QA-8: Monitorability	Medium	Medium	QA-9: Testability	Medium	Medium
Scenario ID	Importance to the Customer	Difficulty of Implementation According to the Architect																													
QA-1: Performance	High	High																													
QA-2: Reliability	High	High																													
QA-3: Availability	High	High																													
QA-4: Scalability	High	High																													
QA-5: Security	High	Medium																													
QA-6: Modifiability	Medium	Medium																													
QA-7: Deployability	Medium	Medium																													
QA-8: Monitorability	Medium	Medium																													
QA-9: Testability	Medium	Medium																													

Category	Details
Constraints	All of the previously discussed constraints are included as drivers.
Architectural concerns	All of the previously discussed architectural concerns associated with the system are included as drivers.

8.4.2 Iteration 1: Establishing an Overall System Structure

This section presents the results of the activities that are performed in each of the steps of ADD in the first iteration of the design process.

8.4.2.1 Step 2. Establish Iteration Goal by Selecting Drivers

This is the first iteration in the design of a greenfield system, so the iteration goal is to achieve concern CRN-1, *establishing an overall system structure*.

Although this first iteration is driven by a general architectural concern, the architect must keep in mind *all* of the drivers that may influence the general structure of the system. In particular, the architect must be mindful of the following:

- QA-1: Performance
- QA-2: Reliability
- QA-3: Availability
- QA-5: Security
- CON-1: Access to the system via web browser
- CON-2: Cloud provider
- CON-6: Develop cloud-native solution
- CRN-2: Leverage team's knowledge about Java technologies
- CRN-4: Avoid introducing technical debt

Accounting for security is particularly important early in the design process so this quality attribute is given special consideration, even though the architect does not consider it to be too difficult to implement.

8.4.2.2 Step 3. Choose Elements of the System to Refine

Since this project involves the complete replacement of an existing system, the first element to refine is the whole HPS system (shown in Figure 8.1). In this case, refinement is performed through decomposition.

8.4.2.3 Step 4. Choose Design Concepts That Satisfy the Selected Drivers

In this initial iteration, given the goal of structuring the entire system, design concepts are selected according to the roadmap presented in Section 4.3.1. Table 8.9 summarizes the selection of design decisions. Also, security is considered from the beginning of the design process. The words in **bold** in the table refer to the design concepts that are selected.

TABLE 8.9 Design Concept Selection Decisions for the Hotel Pricing System: Iteration 1

Design Decision and Location	Rationale
Structure the back-end part of the system following the CQRS pattern and separate the command and query components that communicate through events.	CQRS (Command and Query Responsibility Segregation) is a pattern that separates changes to data from queries to this data. This pattern is useful to support performance, scalability, availability, and security, albeit at the cost of added complexity (see Section 7.3.3.2). The command and query components communicate using events. This type of communication using events also supports the decision by the company to move away from connecting applications using traditional request-response invocations. Furthermore, this will allow other applications to connect to the event channel in the future to perform tasks such as analysis or event storage. Discarded alternatives include developing the back-end as a monolithic application. The pricing system that is being replaced was structured as a monolith. Although this was not the reason for its problems, it is believed that starting with a monolith is not aligned with the enterprise architecture principles of the company and current best practices. Furthermore, if the application is structured as a monolith, scaling it to support future growth will require creating multiple instances of the whole application. Although CQRS could be implemented in a monolith, the separation of the application into different components allows the query side of the application to be scaled independently of the command side.
Implement the command and query components as microservices .	The command and query parts of the system are developed and deployed as independent microservices that expose service APIs, making this a service-oriented application. Multiple replicas of the query side can be set up to support larger query volumes and higher availability. Security is enhanced because clients accessing the query side cannot make changes to important aspects of the system such as business rule calculations. In addition to the benefits in terms of supporting quality attributes, the use of this pattern in this context is appropriate because all prices do not need to be recalculated with each query, and because changes in the calculations of business rules do not affect published prices prior to the date of change of the business rule calculation. The use of microservices also allows CRN-3 to be addressed, as each microservice can be implemented by a small team.
Secure the APIs exposed by the microservices by implementing the authentication and authorization of actors tactics and by implementing the limit access and encrypt data tactics.	The front-end of the system along with external systems will interact with the microservices through APIs. The access to the functionalities provided by the APIs should be secured. Furthermore, access to other resources such as databases needs to be limited because all interactions are made through APIs or through events. All of the information that is exchanged through APIs is encrypted.
Logically structure the client application using the Rich Internet Application reference architecture.	The system must be accessed from a web browser (CON-1). The front-end part of the system is developed as a Rich Internet Application that consumes APIs on the back-end side. Discarded alternatives include developing the application as a rich client or a mobile application.

8.4.2.4 Step 5. Instantiate Architectural Elements, Allocate Responsibilities, and Define Interfaces

The instantiation design decisions considered and made are summarized in Table 8.10.

TABLE 8.10 Instantiation Decisions for the Hotel Pricing System: Iteration 1

Design Decision and Location	Rationale
Use Apache Kafka as the event bus.	Kafka is a durable message broker that enables applications to process, persist, and reprocess streamed data. Events are published and consumed from "topics". Kafka supports message ordering through the use of keys, which is necessary because changes to the hotel prices for a given day must be ordered. Kafka also has the benefit of retaining all of the messages in a log, which acts as the "source of truth" for prices. Discarded alternatives are RabbitMQ and other messaging alternatives that are less familiar to the team (CRN-2).
Create an additional microservice to handle price change events and deliver them to the Channel Management System.	The Channel Management System is a legacy application that cannot be changed to listen to the CQRS events. Instead, an additional microservice is created so that when it receives events from the command side, these events are then pushed to the CMS. This microservice will not require a database, because messages are pulled from the log (Kafka) and sent to the CMS. In case the CMS is not available, messages are put back in the log and delivery is retried. In the future, if the CMS is updated or replaced, its integration with the pricing system will require no changes to the HPS.
Use Angular to implement the front-end component.	Angular is selected because the team has experience with this framework and because it supports the development of responsive front-ends. This is necessary to support CON-1. Discarded alternatives are other web frameworks such as React and Vue.js, mainly because the team is not familiar with them.
Use Docker containers to promote portability across environments.	Because the application has to be moved across environments (QA-7), deploying the application as a set of container images is preferred. Each microservice is deployed in a container. This ensures that the application will run in the same execution environment from development to production. A discarded alternative is the use of virtual machines, as the size of VM images is typically considerably larger than container images; also VMs require more resources (memory, CPU) than containers.
Use TLS (HTTPS) to encrypt data in transit.	All of the data in transit—that is, the data that is exchanged with the front-end and external systems—is encrypted. At this point no decisions have been made regarding data at rest (in the databases).
Secure the API through the use of JSON web tokens .	Access tokens are a proven authentication and authorization mechanism for APIs. These tokens are provided by the User Identity Service once a user has successfully provided their credentials.
Use an API gateway to limit access to the microservices that expose the APIs.	Access to the microservices is not performed directly but rather through an API gateway, which can perform various functions including securing and monitoring the APIs. The calls that are received by the API are routed to the microservices, which are protected inside a private network in the cloud.

The results of these instantiation decisions are recorded in the next step. In this initial iteration, it is typically too early to precisely define functionality and interfaces. In the next iteration, which is dedicated to defining functionality in more detail, interfaces will begin to be defined.

8.4.2.5 Step 6. Sketch Views and Record Design Decisions

Figure 8.4 shows the sketch of a component view of the architecture adapted according to the design decisions we have made.

This sketch is created using an informal representation. A short description of the responsibilities of the elements is also captured. The fact that each component has a specific function promotes high cohesion and thus helps avoid technical debt (CRN-4). Note that the descriptions at this point are quite crude, just indicating major functional responsibilities, with no details. Table 8.11 summarizes the information that is captured.

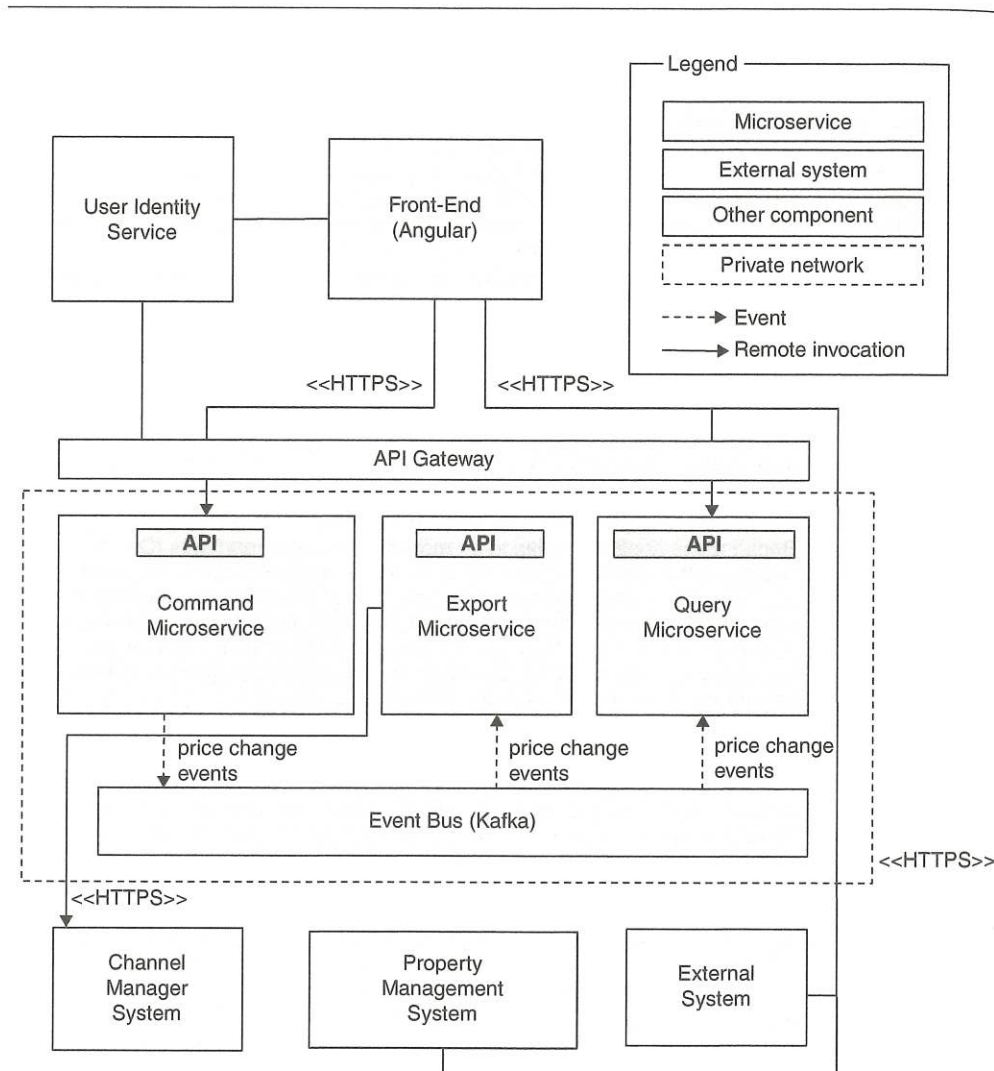


FIGURE 8.4 Main components of the Hotel Pricing System

TABLE 8.11 Elements of the Hotel Pricing System: Iteration 1

Element	Responsibility/Description
Front-End	Represents the client-side application built using the Rich Internet application reference architecture and implemented using Angular.
API Gateway	The API gateway sits between the clients of the API and the microservices that expose them.
Command Microservice	This microservice encapsulates the logic for the command part of the CQRS pattern. This includes managing the different hotels, rates, room types, and price calculation business rules.
Query Microservice	This microservice's primary responsibility is to serve price requests. It exposes two APIs: one used by the front-end and one used by legacy internal systems.
Export Microservice	This microservice's primary responsibility is to consume price change events and publish them on the Channel Management System.
Event Bus	This event bus is used by the Command microservice to publish price change events, and by the Query and Export microservices to consume these events.
Property Management System	This element represents the legacy Property Management System, which needs to query prices.
Channel Management System	This element represents the legacy Channel Management System, which must be notified of changes in the prices. The Export microservice is in charge of notifying this system.
External System	This element represents any other system that needs to query prices.
User Identity Service	Service from the cloud provider that manages users and provides access tokens for the APIs.

In this iteration, initial decisions about how to deploy the application are made. The microservices and the event bus are deployed inside a private network and are only accessible through the API gateway. Decisions about specific cloud resources to be used still need to be made.

8.4.2.6 Step 7. Perform Analysis of the Current Design and Review the Iteration Goal and Achievement of Design Purpose

The decisions made during this iteration result in new concerns that need to be addressed:

- CRN-6: Select specific cloud technologies

Table 8.12 summarizes the design progress using the Kanban technique discussed in Section 4.8.2.

TABLE 8.12 Design Progress for the Hotel Pricing System: Iteration 1

Not Addressed	Partially Addressed	Completely Addressed	Design Decisions Made During the Iteration
	HPS-1		Login will be performed using the cloud provider identity management service, but additional decisions need to be made.
	HPS-2		The overall structure of the application was decided based on the need to support this use case, but many details have not yet been addressed.

continues

Not Addressed	Partially Addressed	Completely Addressed	Design Decisions Made During the Iteration
	HPS-3		The use of CQRS and the creation of a specific microservice to support queries was made to support this use case, but many details have not yet been addressed.
HPS-4			No relevant decisions made yet.
HPS-5			No relevant decisions made yet.
HPS-6			No relevant decisions made yet.
	QA-1		Separation of command and query sides will promote performance, but many details have not yet been addressed.
	QA-2		The selection of Kafka, a highly reliable message broker, supports this scenario, but many decisions (e.g., the configuration of Kafka itself) have not yet been addressed.
	QA-3		The use of CQRS, Kafka, microservices, and a container management service in the cloud provider support this scenario, but many details have not yet been addressed.
	QA-4		Scalability on the query side will be supported by creating multiple replicas of the Query microservice. Additional decisions to make this microservice scalable need to be made.
	QA-5		Use of an API gateway, HTTPS, and access tokens contribute to the satisfaction of this driver.
QA-6			No relevant decisions made yet.
	QA-7		The use of container technologies is a starting point to support this driver, but additional decisions need to be made.
QA-8			No relevant decisions made yet.
		CON-1	The front-end will run in a web browser; the Angular framework is well supported by modern browsers.
		CON-2	While this constraint has been considered, a new concern has appeared (CRN-6) regarding the selection and configuration of products from the cloud provider.
CON-3			No relevant decisions made yet.
	CON-4		Initial decisions, such as the identification of microservices, that can help estimate development time have been made, but additional information is needed.
CON-6			No decisions about specific cloud provider resources have been made yet.
		CRN-1	Establishing an overall initial system structure was the goal of this iteration.
	CRN-2		Decisions regarding the implementation of the microservices have not yet been made, but Angular has been chosen as the implementation framework for the front-end.
		CRN-3	The decomposition of the system into different microservices will allow work to be allocated to members of the development team (one small team per microservice).

Not Addressed	Partially Addressed	Completely Addressed	Design Decisions Made During the Iteration
	CRN-4		Microservices have specific functions, promoting high cohesion and modifiability.
CRN-5			No relevant decisions made yet.
CRN-6			No relevant decisions made yet.

8.4.3 Iteration 2: Identifying Structures to Support Primary Functionality

This section presents the results of the activities that are performed in each of the steps of ADD in the *second* iteration of the design process for the HPS. In this iteration, we move from the relatively abstract view of the architecture developed in iteration 1 to more detailed decisions that will drive implementation.

This movement from the generic to the specific is intentional, and is built into the ADD method. We can't design everything up front, so we need to be disciplined about which decisions we make, and when, to ensure that the design is done in a systematic way, addressing the biggest risks first and moving from there to ever finer details. Our goal for the first iteration was to establish an overall system structure. Now that this goal has been met, our goal for this second iteration is to reason about the units of implementation, which affect team formation, interfaces, and how development tasks may be distributed, outsourced, and implemented in sprints.

8.4.3.1 Step 2. Establish Iteration Goal by Selecting Drivers

The goal of this iteration is *identifying structures to support primary functionality*. Identifying these elements is useful for understanding how functionality is supported and in providing a better estimate for development.

In this second iteration, the architect considers the system's primary user stories:

- HPS-2: Change prices
- HPS-3: Query prices

Note that in this iteration, the design is focused on the back-end part of the application.

8.4.3.2 Step 3. Choose Elements of the System to Refine

The elements that will be refined in this iteration are the Command, Query, and Export microservices derived from the CQRS pattern used in the first iteration.

8.4.3.3 Step 4. Choose Design Concepts That Satisfy the Selected Drivers

Table 8.13 summarizes the design decisions made. The words in **bold** refer to the design concepts that are selected.

TABLE 8.13 Design Concept Selection Decisions for the Hotel Pricing System: Iteration 2

Design Decision and Location	Rationale
Create a domain model for the application.	Before starting a functional decomposition, it is necessary to create an initial domain model for the system, including identifying the major entities in the domain and their relationships. While this is not an architectural decision per se, it is important and interacts with all subsequent architectural choices. There are no shortcuts or good alternatives for this step. A domain model must eventually be created, or it will emerge in a suboptimal fashion leading to an ad hoc architecture that is hard to understand and maintain.
Logically structure the microservices using the Service Application reference architecture. Microservices expose APIs.	Service applications (see Section 3.2.1) do not provide a user interface but rather expose services that are consumed by other applications. This reference architecture is suitable to logically structure each microservice that exposes an API. These APIs will be consumed by the client application and by other legacy systems (CON-5). There are no discarded alternatives with respect to the reference architecture.

8.4.3.4 Step 5. Instantiate Architectural Elements, Allocate Responsibilities, and Define Interfaces

The instantiation design decisions made in this iteration are summarized in Table 8.14.

TABLE 8.14 Instantiation Decisions for the Hotel Pricing System: Iteration 2

Design Decision and Location	Rationale
Create a domain model for the command-side microservice.	The domain model that is to be created is not a domain model for the entire company, but rather a model that is useful in the context of hotel prices. This model resides on the command-side microservice. The query- and export-side microservices do not have their own specific domain models, as there was not a compelling reason to create them. In the case of the query side, price change events are stored directly in the data store. In the case of the export side, no information is stored (as we will see later).
Create a PriceChangeEvent and use a single topic and Kafka keys to ensure temporal ordering of price changes.	Kafka was chosen in the previous iteration, and in iteration 2 further decisions are made: ▪ A single topic is used for price change events; these events contain the prices for a given hotel and date. ▪ A key that guarantees price ordering is defined. This key allows log compaction to be supported.
Use a relational database for the Command microservice.	Given that the Command microservice manages the entities of the domain model (e.g., hotel prices, rates, room types), and given that these entities are related to each other, the most natural and appropriate mechanism for storing them is a relational database. Discarded alternatives are nonrelational databases and other types of storage.

Design Decision and Location	Rationale
Use a nonrelational database for the Query microservice.	Because the database on the query side does not need to handle transactions and relationships between entities, and because the prices that are stored have a variable structure depending on the hotel, it was decided that the use of a nonrelational database is more appropriate. PriceChangeEvent can be appropriately stored in a document database. Discarded alternatives include other types of NoSQL, relational databases, and other types of storage.
Use the Spring Framework technology stack to implement the microservices and map the system user stories to the Service Application reference architecture modules .	The Spring Framework technology stack is selected because the team is familiar with it (CRN-2). Because of this, no alternative frameworks for languages other than Java are considered. The Service Application reference architecture and Spring Framework provide categories of modules for the different layers: controllers to expose APIs, services to manage business logic, entities to manage business entities, and repositories to persist them, among others. User stories can typically be mapped to modules following well-established rules: One entity can be mapped to at least one controller, one service, and one repository. This technique ensures that modules that support all of the functionalities and entities are identified. The architect will perform this task only for the primary use cases. This allows another team member to identify the rest of the modules to allocate work among team members (CRN-3). Having established the set of modules, the architect realizes the need to test these modules, so a new architectural concern is identified here: ▪ CRN-7: Ensure that all non-generated modules can be unit tested The reason that only “non-generated” modules are covered by this concern is that some modules, such as repositories, are generated by Spring and do not need to be unit tested. The use of Spring facilitates testability because it is built upon the Inversion of Control (IoC) pattern.
Define and expose REST APIs to the front-end and document them using Swagger/OpenAPI .	The API exposed by the microservices for consumption by the front-end will follow the REST paradigm and exchange JSON data. REST is selected because it is commonly used to communicate with Rich Internet front-ends; in this case, there are no requirements that would constrain the use of this paradigm. Furthermore, JSON is selected as the data exchange format for similar reasons. A new concern arises from this decision: ▪ CRN-8: Document the API This new concern is immediately addressed by selecting Swagger/OpenAPI as the framework used to document the API. Swagger is selected because it is a mature framework that integrates easily with the Spring Framework.
Apply the externalize configurations tactic to promote portability.	To support moving from one execution environment to another without changing code or repackaging, configuration is externalized and environment variables are used to configure aspects such as database connections.
Use cloud provider managed services for databases, container orchestration, and Kafka.	After evaluating the alternative products available from the cloud provider and considering factors such as cost, licensing, ease of installation, and administration, a decision was made to select products directly managed by the cloud provider for the databases, container orchestration, and Kafka.

continues

Design Decision and Location

There are no anticipated needs to port this application to a different cloud provider once deployed, so open source solutions that can be deployed on the cloud but that require more management are discarded. Although these solutions could be cheaper than managed services, the time required to set them up is considerable and an economic analysis of the costs of managed services has convinced management that they are acceptable (CON-6). Although the products are now selected, configuring them adequately is still pending. A new concern is raised:

- CRN-9: Configure managed cloud services

Although the structures and interfaces are identified in this step of the ADD method, they are only captured in the next step. For this reason, they are not shown here.

8.4.3.5 Step 6. Sketch Views and Record Design Decisions

Figure 8.5 shows an initial domain model to be used in the command-side microservice.

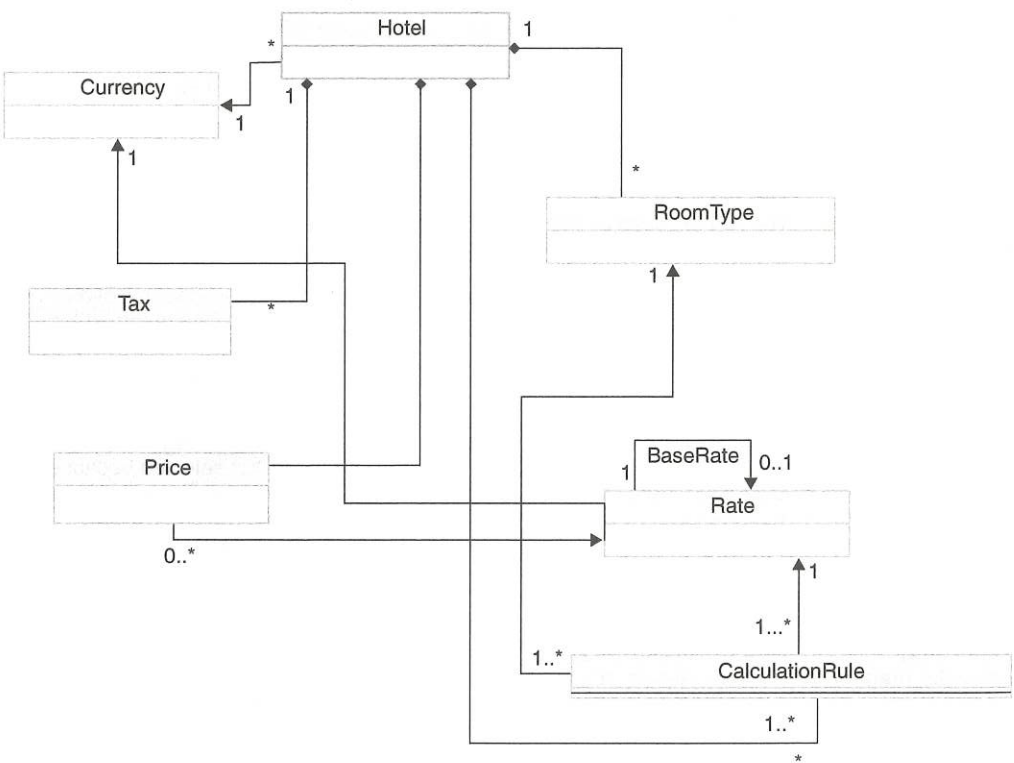


FIGURE 8.5 Domain model for the command side of the Hotel Pricing System (Key: UML)

The entities of the domain model are described in Table 8.15.

TABLE 8.15 Elements of the Domain Model for the Hotel Pricing System

Element	Responsibility/Description
Hotel	A hotel, which is the main entity.
Currency	The primary currency associated with the hotel.
RoomType	A particular type of room. One hotel may have different types of rooms.
Tax	Taxes associated with the hotels.
Rate	Different room rates that are used to calculate prices.
CalculationRule	Rules that are applied to rates to calculate prices.
Price	The price in the base rate (or a fixed rate) for a particular date.

Figure 8.6 shows a sketch of a module view of the command-side microservice with modules derived from the entities that are associated with the primary use case (HPS-2).

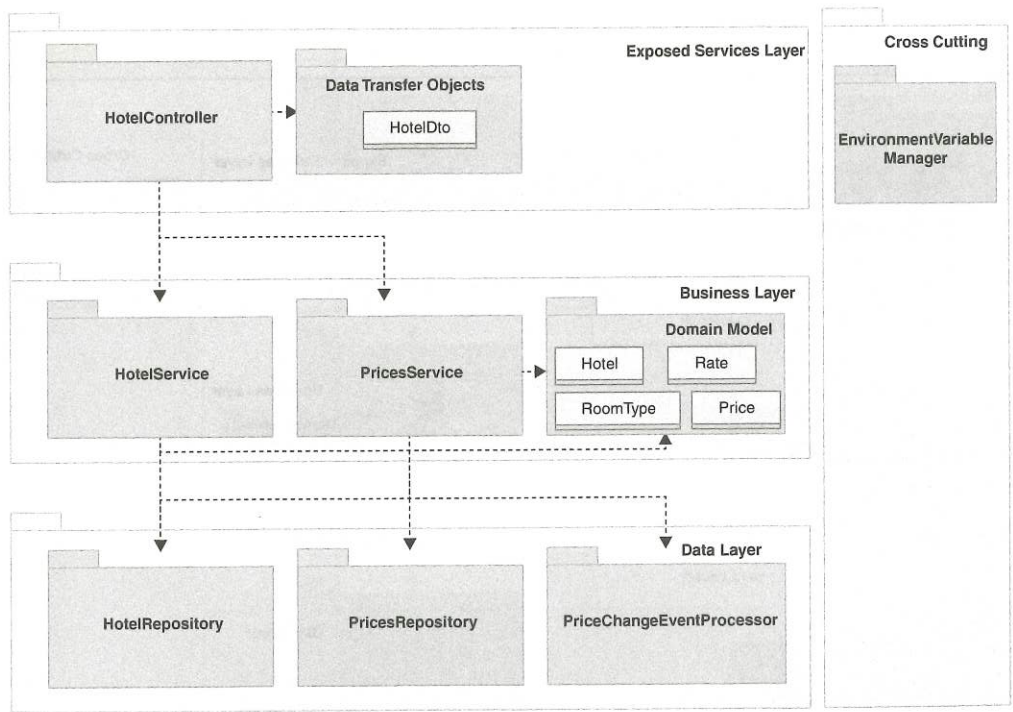


FIGURE 8.6 Module view of the command-side microservice (Key: UML)

The responsibilities for the elements identified in Figure 8.6 are summarized in Table 8.16.

TABLE 8.16 Responsibilities of the Command-Side Microservice Modules

Element	Responsibility/Description
HotelController	Exposes API endpoint for CRUD operations for the Hotel resource and contains annotations that produce Swagger documentation.
DataTransferObjects	Contains objects that are returned or received by the API endpoint.
HotelService	Validates certain business rules that do not fit inside entities and coordinates objects from the domain model and data layer to support operations exposed by the controller. Also manages certain transactional operations.
PricesService	Coordinates price changes.
Domain Model	Contains the business entities. These entities are "rich" in the sense that they contain business logic.
HotelRepository	Manages persistence of Hotel objects.
PricesRepository	Stores public rate prices for given days. Prices for other rates are not stored because they are calculated and sent as events to the query side.
PriceChangeEvent-Processor	Generates events related to price changes.
Environmental-VariableManager	Manages externalized configuration values (environment variables).

Figure 8.7 shows a sketch of a module view of the query-side microservice.

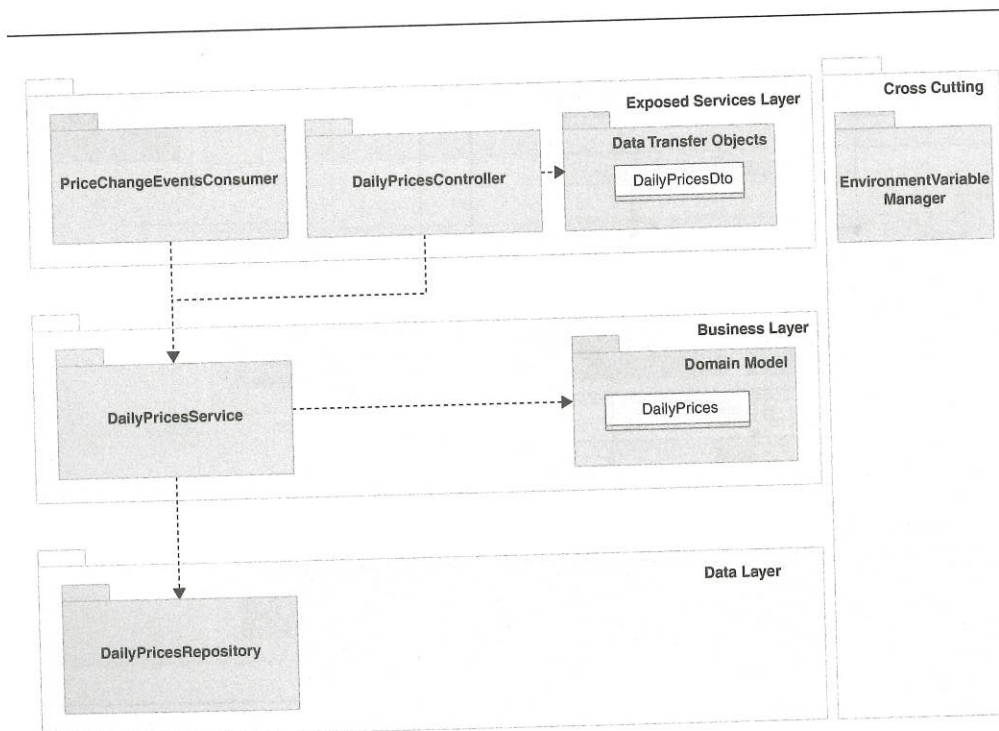


FIGURE 8.7 Module view of the query-side microservice (Key: UML)

The responsibilities of the elements identified in Figure 8.7 are summarized in Table 8.17.

TABLE 8.17 Responsibilities of the Query-Side Microservice Modules

Element	Responsibility/Description
PriceChangeEventConsumer	Receives price change events that are published in Kafka.
DailyPricesController	Exposes APIs to query prices.
DailyPricesService	Prepares price change events to be stored.
DailyPricesRepository	Stores events in the (nonrelational) database.
Data Transfer Objects	Prices that are returned as JSON or XML objects.
Domain Model	Contains a class that stores the information from the price change events.
EnvironmentalVariableManager	Manages externalized configuration values (environment variables).

Figure 8.8 shows a sketch of a module view of the export-side microservice.

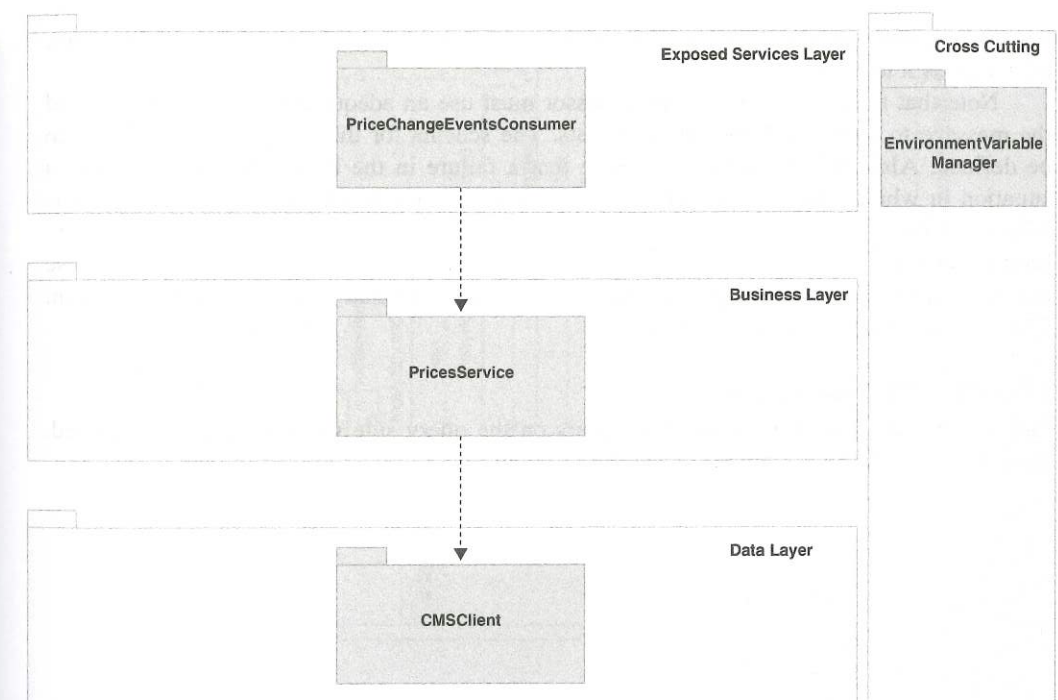


FIGURE 8.8 Module view of the export-side microservice (Key: UML)

The responsibilities of the elements identified in Figure 8.8 are summarized in Table 8.18.

TABLE 8.18 Responsibilities of the Export-Side Microservice Modules

Element	Responsibility/Description
PriceChangeEventsConsumer	Receives price change events that are published in Kafka.
DailyPricesService	Prepares payload to be sent to the CMS.
CMSClient	Communicates with the CMS.
EnvironmentalVariableManager	Manages externalized configuration values (environment variables).

Next, we show the sequence diagrams for HPS-2, which are used to define interfaces (as discussed in Section 4.6).

8.4.3.5.1 HPS-2 Command Side

Figure 8.9 shows an initial sequence diagram for HPS-2 (Change prices) on the command side. The client application invokes the POST method that triggers a change in prices. The PriceService retrieves the hotel and price that must be changed, and asks the hotel object to calculate the additional types of prices. To increase performance, an index is added to the database. Once prices are calculated, an event is created and sent to the PriceChangeEventProcessor, which sends it to Kafka.

Note that the PriceChangeEventProcessor must use an adequate topic and keys to send the message to ensure ordering of the events. The schema for the event payload remains to be defined. Also, it is necessary to ensure that a failure in the broker does not result in a situation in which a price is modified but an event is not sent (the method is annotated as @Transactional, but configuration of the transaction manager is pending). Finally, the event broker must be configured to support adequate performance and reliability. All of these aspects can be considered as part of the additional concern that is created in this iteration: CRN-9: Configure managed cloud services.

8.4.3.5.2 HPS-2 Query Side

Figure 8.10 shows the interaction that occurs on the query side when the event is received. Here, the event payload is simply stored in the database.

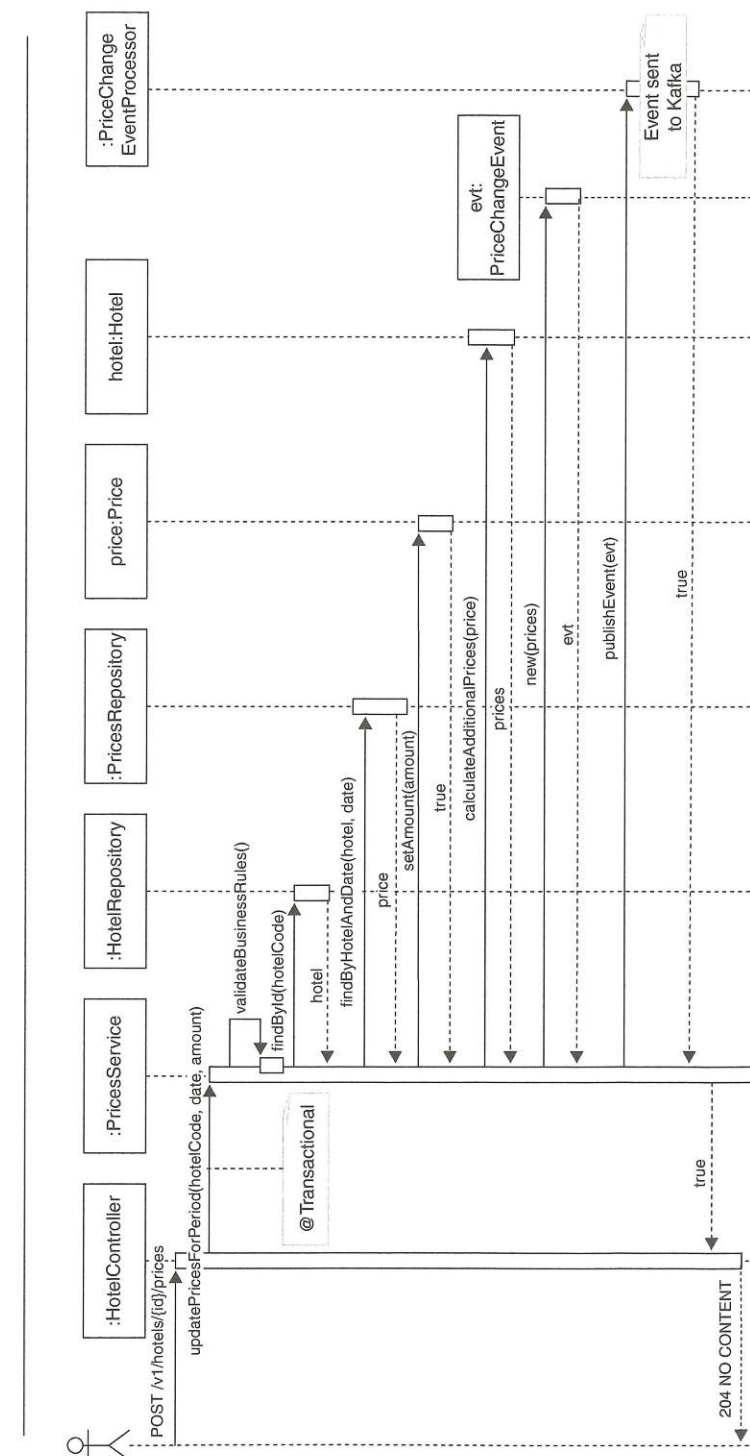


FIGURE 8.9 Sequence diagram for HPS-2 (Change prices) on the command side (Key: UML)

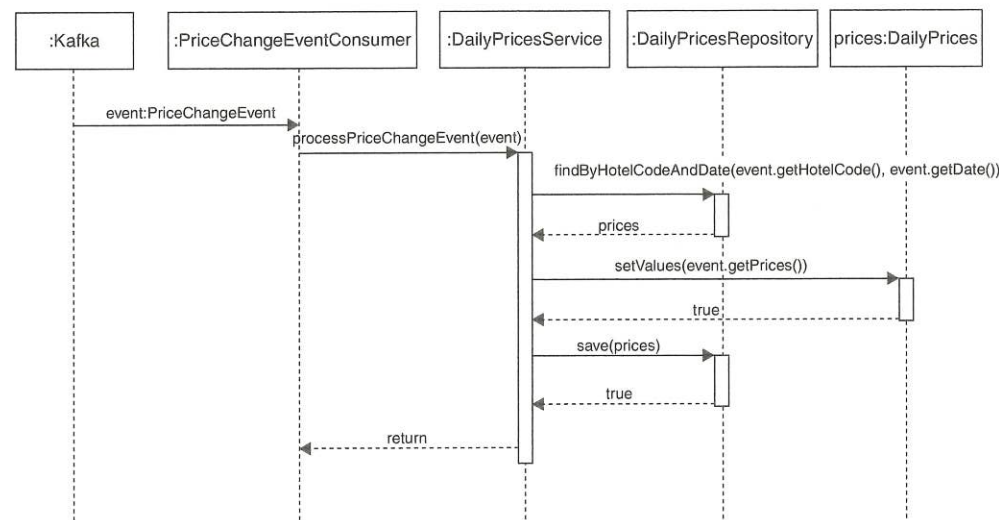


FIGURE 8.10 Sequence diagram illustrating the reception of events in the query side (Key: UML)

8.4.3.5.3 HPS-2 Export Side

Figure 8.11 shows the interaction that occurs on the export side when the event is received. In this scenario, the event is consumed and the event contents are used to prepare the payload that is sent to the CMS. After the event is sent, a success message is returned to Kafka.

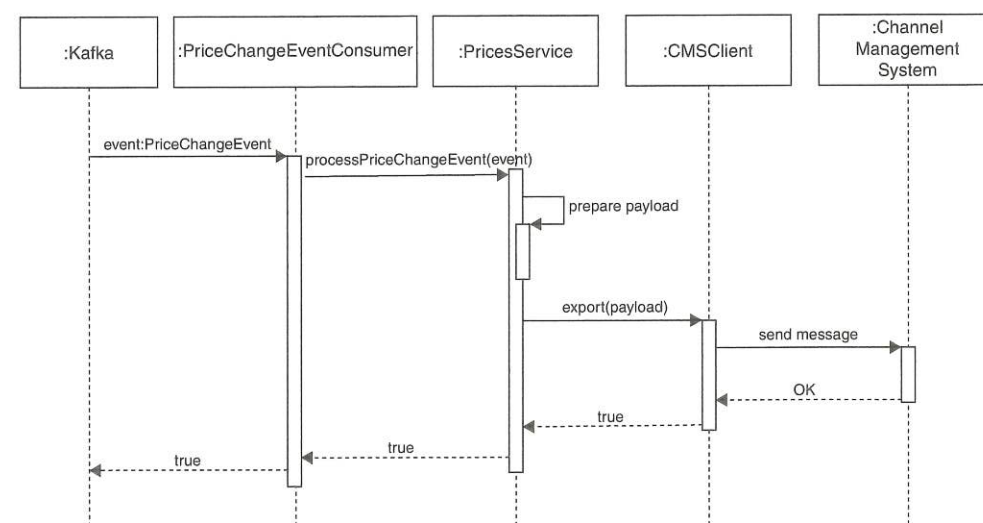


FIGURE 8.11 Sequence diagram illustrating the reception of events in the export side (Key: UML)

From the interactions illustrated in the sequence diagrams, the initial interfaces can be defined. In this case, both an external interface (API) and internal interfaces are identified. Regarding the API, endpoint paths are defined from the domain model. Likewise, API versioning (an additional concern) is considered in the name of the path. A more detailed description of the methods will eventually need to be established, but here we provide a summary of only the most relevant methods.

8.4.3.5.4 HPS-2 Interfaces

HotelController

Method Name	Description
POST /v1/hotels/{id}/prices	Allows prices for a hotel to be updated; returns HTTP NO CONTENT if change is successful, or HTTP BAD REQUEST if parameters are incorrect. Parameters: - Hotel Identifier - Range of dates of price changes - Amount

The following interfaces from the modules provide a basis for defining internal interfaces. Only a sample of methods from the command side are described here.

PriceServices

Method Name	Description
updatePriceForPeriod	Update prices for a hotel and produce a price change event. Parameters: - Hotel Identifier - Range of dates of price changes - Amount Returns: - Boolean: true if successful, false otherwise Throws: - PriceChangeEventException in case price change fails.

PriceRepository

Method Name	Description
findByHotelAndDate	Retrieves a price object associated with a hotel and a particular date. Parameters: - Hotel object - Date Returns: - Price object or null

Hotel

Method Name	Description
calculateAdditionalPrices	Calculates prices associated with all the rates and room types for this hotel. Returns a data structure with the prices. Parameters: ▪ Price for base room Returns: ▪ Prices for other rooms and rates

PriceChangeEventProcessor

Method Name	Description
publishEvent	Publishes an event to the broker for the appropriate topic with a key that ensures correct ordering. Parameters: ▪ PriceChangeEvent Returns: ▪ Prices for other rooms and rates

In regard to deployment, the design decisions have resulted in a clearer vision of the solution in the cloud environment. At this point, specific products from the cloud provider have been selected, but they still need to be configured appropriately to support the quality attribute goals (CRN-9). Figure 8.12 presents an allocation view of the system using cloud-managed resources.

The responsibilities of the elements identified in Figure 8.12 are summarized in Table 8.19.

TABLE 8.19 Deployment Elements for the Hotel Pricing System: Iteration 2

Element	Responsibility/Description
User Identity Management Service	Manages users; provides and validates access tokens used to access APIs.
API Gateway	Restricts direct access to the microservices and secures the APIs. Also responsible for other aspects such as monitoring.
Load Balancer	Forwards and balances the requests to the microservices inside the private cloud network.
Container Orchestration Service	Provides an execution environment for the containers associated with the different microservices. This service also monitors the health of the containers and, if needed, restarts them in case of failures, contributing to improved availability (QA-3).
Managed Relational Database	Database service managed by the cloud provider.
Managed Nonrelational Database	Database service managed by the cloud provider.
Managed Kafka Service	Kafka service managed by the cloud provider.

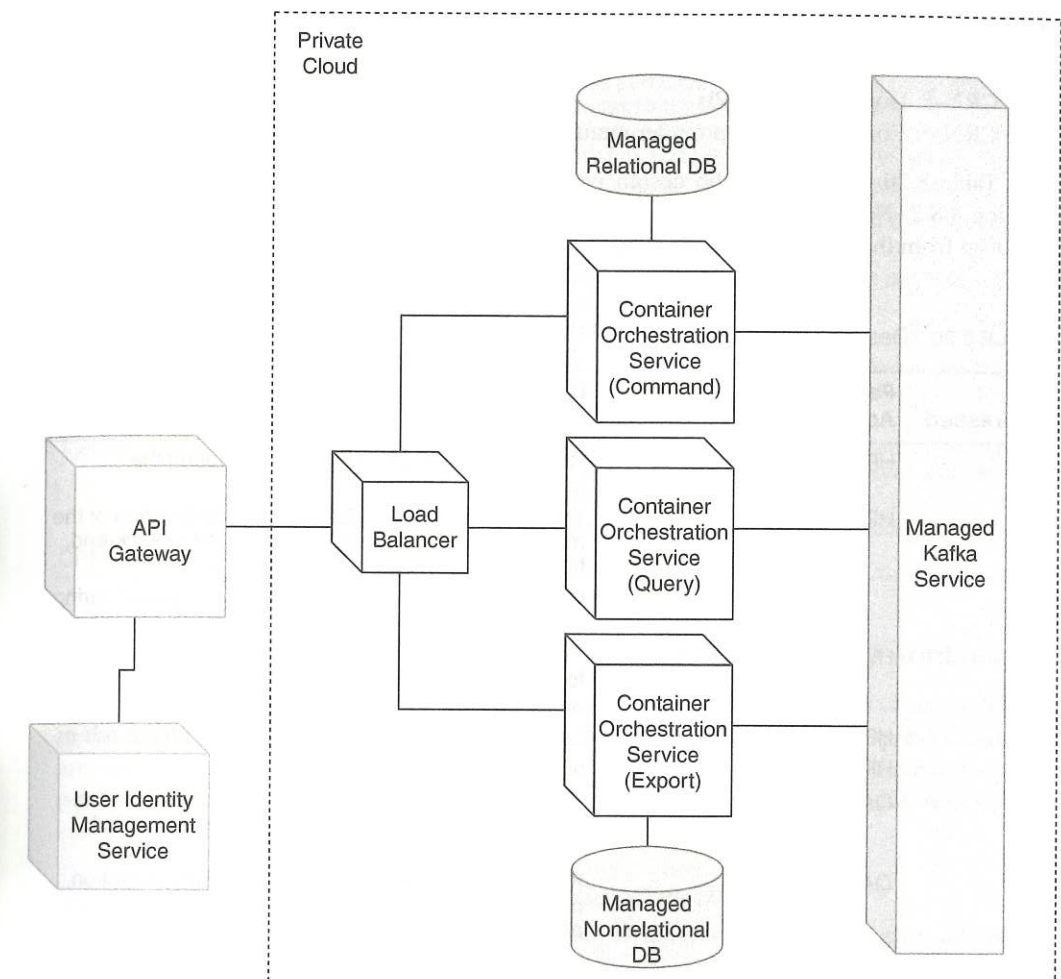


FIGURE 8.12 Initial deployment diagram of the system (Key: UML)

8.4.3.6 Step 7. Perform Analysis of the Current Design and Review the Iteration Goal and Achievement of Design Purpose

The decisions made in iteration 2 provided an initial understanding of how functionality is supported in the system. Furthermore, the design decisions made at this point are sufficient to produce a Minimum Viable Product (MVP) for the system, which can be demonstrated to internal stakeholders and which supports price changes and queries to these changes (CON-4).

New architectural concerns have also been added as a result of the decisions made in this iteration:

- CRN-7: Ensure that all non-generated modules can be unit tested
- CRN-8: Document the APIs
- CRN-9: Configure cloud provider managed services

Table 8.20 summarizes the design progress using the Kanban technique discussed in Section 4.8.2. Note that drivers that were completely addressed in the previous iteration are removed from the table.

TABLE 8.20 Design Progress for the Hotel Pricing System: Iteration 2

Not Addressed	Partially Addressed	Completely Addressed	Design Decisions Made During the Iteration
	HPS-1		No additional decisions were made during the iteration.
	HPS-2		This iteration was dedicated to the identification of the modules that support this use case on the back-end, but the front-end part is not yet designed.
		HPS-3	Querying through the API has been addressed during this iteration.
	HPS-4		No relevant decisions have been made yet, but rules for module identification used for HPS-2 can also be applied for this use case.
	HPS-5		same as for HPS-4
	HPS-6		same as for HPS-4
	QA-1		Adding an index to the database will accelerate the retrieval of the information required by HPS-2, but additional decisions need to be made.
	QA-2		Kafka is highly reliable, and the use of a transaction manager to ensure that an event is sent when prices are changed will promote reliability.
	QA-3		No additional decisions were made during this iteration.
	QA-4		No additional decisions were made during this iteration.
	QA-5		Use of an API gateway, HTTPS, and access tokens contribute to the satisfaction of this driver.
		QA-6	The way the Query microservice is structured, based on the Service Application reference architecture, will help add other types of query endpoints.
	QA-7		The use of containers and externalized configurations (through <code>EnvironmentalVariableManager</code>) should allow this requirement to be satisfied. There are still some decisions to be made regarding the way configurations will be externalized and how containers will be built.
QA-8			No relevant decisions made yet.
CON-3			No relevant decisions made yet.

Not Addressed	Partially Addressed	Completely Addressed	Design Decisions Made During the Iteration
		CON-4	The level of decomposition achieved in this iteration provides sufficient information to estimate and understand if the time constraint can be supported. The result of this iteration can already support the primary functionality and can be demoed.
		CON-6	Cloud-managed resources have been selected.
		CRN-2	The Spring framework was selected to implement the microservices.
	CRN-4		The decisions made in this iteration promote high cohesion and modifiability.
CRN-5			No relevant decisions made yet.
	CRN-6		Specific products from the cloud provider have been identified. Other products will need to be selected as design progresses.
		CRN-7	The use of Spring, which is based on the IoC pattern, facilitates unit testing using frameworks such as JUnit and Mockito.
		CRN-8	Swagger/OpenAPI was chosen to document the APIs.
CRN-9			No relevant decisions made yet.

8.4.4 Iteration 3: Addressing Reliability and Availability Quality Attributes

This section presents the results of the activities that are performed in each of the ADD steps in the third iteration of the design process. While the decisions made in the previous iteration are sufficient to support price changes and querying these price changes from a functional perspective on the back-end side, several quality attributes have not yet been addressed. These are the focus of this iteration.

8.4.4.1 Step 2. Establish Iteration Goal by Selecting Drivers

The goal of this iteration is to identify additional structures to support quality attributes—particularly reliability, availability, and scalability:

- QA-2: Reliability—Guaranteeing that 100% of the price changes are published and exported successfully
- QA-3: Availability—Uptime SLA
- QA-4: Scalability—Increase query volume without decreasing average latency

In the previous iteration, a new concern was identified. This concern plays an important role with respect to the quality attributes:

- CRN-9: Configuration of cloud provider managed services

8.4.4.2 Step 3. Choose Elements of the System to Refine

For this iteration, the elements that are refined are mainly located in the infrastructure part of the solution. Furthermore, additional refinements are made to the export microservice to support failure scenarios.

8.4.4.3 Step 4. Choose Design Concepts That Satisfy the Selected Drivers

The design concepts used in this iteration are summarized in Table 8.21.

TABLE 8.21 Design Concept Selection Decisions for the Hotel Pricing System: Iteration 3

Design Decision and Location	Rationale
Apply tactics for performance: increase resources and maintain multiple copies of computations and data .	Replication is an essential strategy to address both reliability and performance.
Apply tactics for availability: exception handling .	When one of the microservices fails, the problem needs to be detected and addressed so that the system as a whole continues operating normally.
Apply tactics for availability: redundant spare .	There may be a more generalized failure occurring in the cloud's availability zone where the various microservices are located. This situation also needs to be addressed.

8.4.4.4 Step 5. Instantiate Architectural Elements, Allocate Responsibilities, and Define Interfaces

The instantiation design decisions are summarized in Table 8.22.

TABLE 8.22 Instantiation Decisions for the Hotel Pricing System: Iteration 3

Design Decision and Location	Rationale
Use Kafka as the data storage mechanism instead of a database for the Export microservice.	As described in iteration 2, price change events that are received by the export microservice are forwarded to the CMS. In case of communication failures with the CMS, there is no need to store the events to later retry: Kafka can be used as the storage mechanism (this is described in more detail in step 6). A discarded alternative is to add a database to the Export microservice, but this would incur additional costs and would not provide additional benefits.
Configure the container orchestration service to restart microservices in case of failures .	Container orchestration services from the cloud provider support restarting containers in case a failure is detected. A discarded alternative is to use an open source solution to orchestrate containers. However, it was previously decided that managed services are preferred for this project.
Establish an infrastructure configuration to support passive redundancy .	The design of the system allows it to be replicated. This involves creating a duplicate of the infrastructure hosted in a different availability zone or region. When a failure is detected, this replica needs to be started. Configurations of the managed services that support redundancy are therefore selected. A discarded alternative is to manually replicate the whole infrastructure in different availability zones or regions. However, the work involved in setting up and managing such an approach makes it undesirable.
Replicate the Query service.	To increase performance, the Query microservice is replicated and requests are load balanced.

The results of these instantiation decisions are recorded in the next step.

8.4.4.5 Step 6. Sketch Views and Record Design Decisions

Several decisions are made during this iteration to address the drivers listed in step 2. Here we summarize these decisions.

Communication Failure with the Channel Management System

In this iteration, a scenario involving failed communication between the export microservice and the CMS is considered (associated with QA-2). Figure 8.13 shows the interaction that occurs with the Export microservice when the event is received. In this scenario, the event is sent to the CMS but the invocation does not succeed (e.g., there is a timeout). When this situation occurs, the event that was consumed from the event log is sent back to Kafka. This means that the next time the export microservice polls the event log, it will recover the event that was not sent successfully.

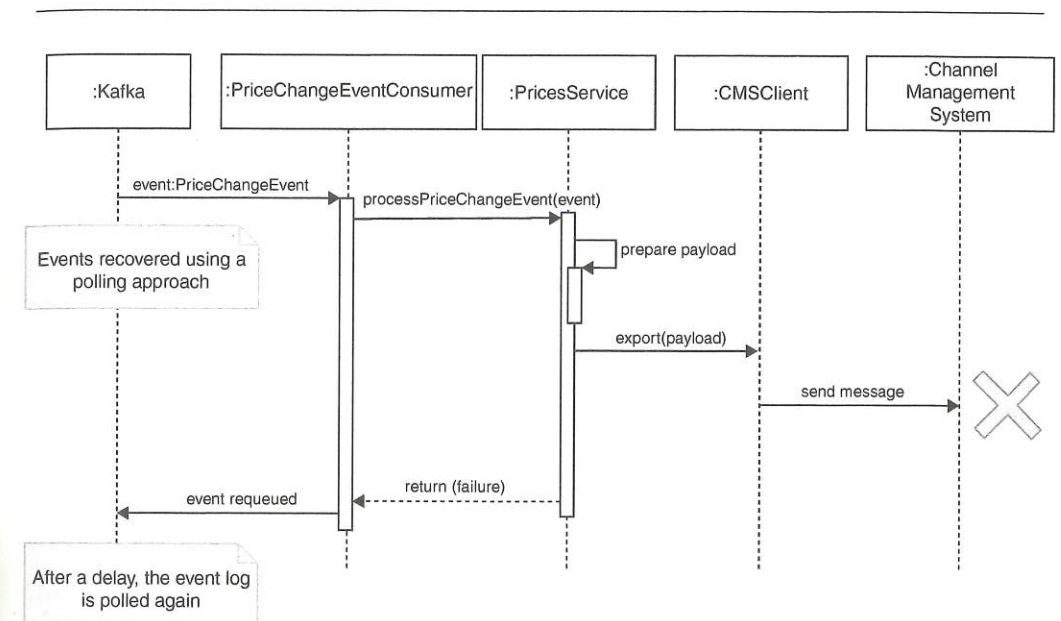


FIGURE 8.13 Sequence diagram of failure scenario on the export side (Key: UML)

Changes in the Infrastructure to Support Quality Attributes

Figure 8.14 shows the managed services with the introduction of replication.

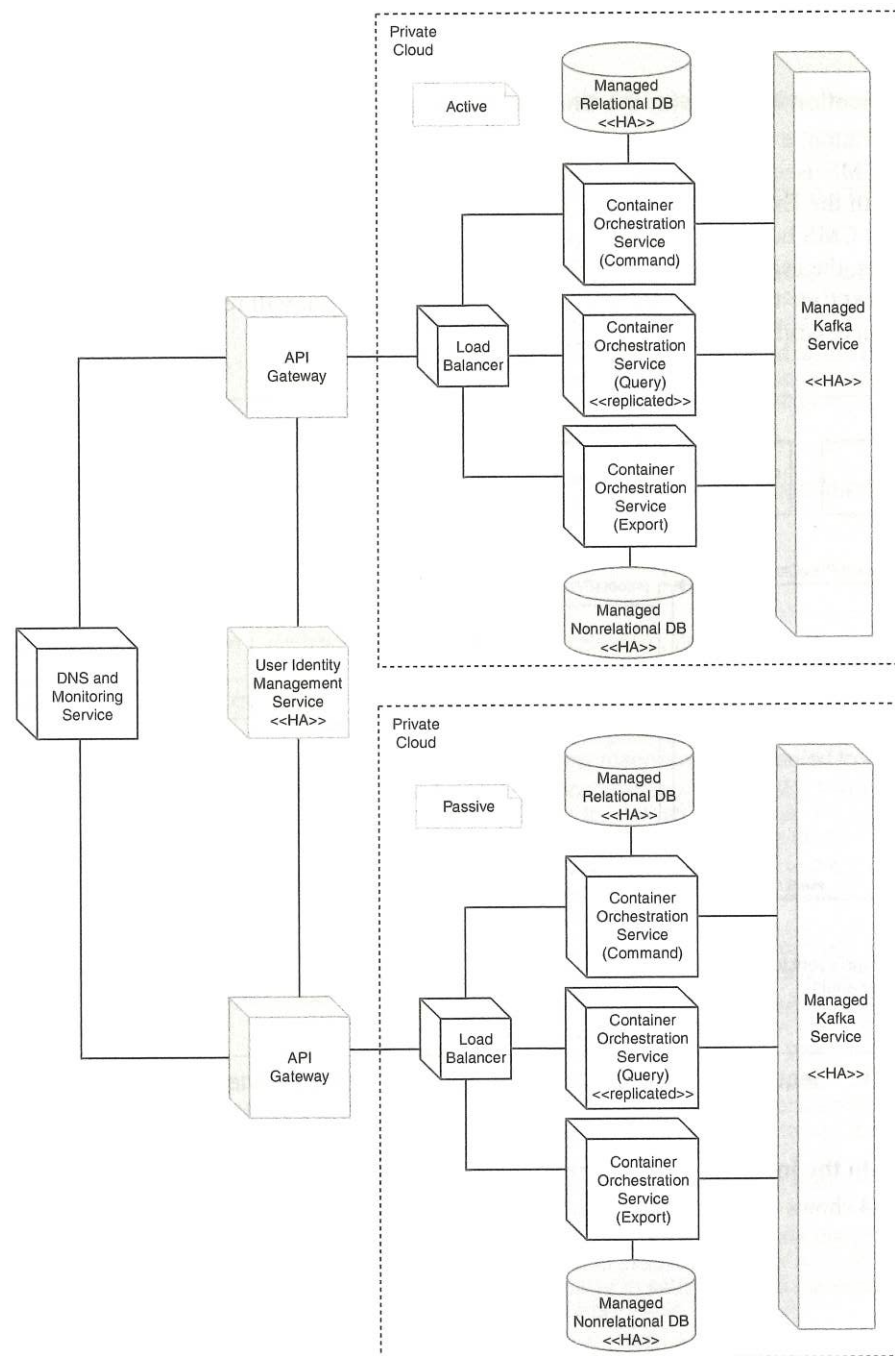


FIGURE 8.14 Refined deployment diagram (Key: UML)

The responsibilities for the elements identified in the previous diagram are summarized in Table 8.23 (only the elements that were updated are described).

TABLE 8.23 Deployment Elements for the Hotel Pricing System: Iteration 3

Element	Responsibility/Description
DNS and Monitoring Service	Monitor the availability of the services in the active replica. If a failure is detected, start up the containers on the passive replica and redirect traffic to the passive replica.
User Identity Management Service	Manage users, and provide and validate access tokens used to access APIs. A high availability configuration is selected where the service is replicated across availability zones.
Container Orchestration Service	Provides an execution environment for the containers associated with the different microservices. This service also monitors the health of the containers and, if needed, restarts them in case of failures, contributing to improved availability (QA-3).
Managed Relational Database	A database service managed by the cloud provider. A high availability configuration is selected where the database is replicated across availability zones.
Managed Nonrelational Database	A database service managed by the cloud provider. A high availability configuration is selected where the database is replicated across availability zones.
Managed Kafka Service	A Kafka service managed by the cloud provider. A high availability configuration is selected where Kafka is replicated across availability zones.

8.4.4.6 Step 7. Perform Analysis of the Current Design and Review the Iteration Goal and Achievement of Design Purpose

Table 8.24 summarizes the status of the different drivers and the decisions that were made during iteration 3. Note that drivers that were completely addressed in the previous iteration have been removed from the table.

TABLE 8.24 Design Progress for the Hotel Pricing System: Iteration 3

Not Addressed	Partially Addressed	Completely Addressed	Design Decisions Made During the Iteration
	HPS-1		No additional decisions were made during the iteration.
	HPS-2		No additional decisions were made during the iteration.
	HPS-4		No additional decisions were made during the iteration.
	HPS-5		No additional decisions were made during the iteration.
	HPS-6		No additional decisions were made during the iteration.
	QA-1		Replication of the Query microservice improves performance and scalability. Tests to validate the measures associated with the quality attributes remain to be done.
	QA-2		Kafka is highly reliable, and the use of a transaction manager to ensure that an event is sent when prices are changed will promote reliability.

continues

Not Addressed	Partially Addressed	Completely Addressed	Design Decisions Made During the Iteration
	QA-3		Container orchestration addresses failures in containers; replication across availability zones addresses failures in an availability zone.
	QA-4		Tests to validate the measures associated with the quality attributes remain to be done.
	QA-5		No additional decisions have been made.
	QA-7		No additional decisions have been made.
QA-8			No relevant decisions made yet.
CON-3			No relevant decisions made yet.
	CRN-4		A careful analysis considering growth scenarios and cost helps to select technologies that are more future-proof, thereby avoiding technical debt.
CRN-5			No relevant decisions made yet.
	CRN-6		Specific products from the cloud provider have been identified. Other products will need to be selected as the design progresses.
	CRN-9		Initial configurations have been established for the container orchestration service and other managed services (database and Kafka).

8.4.5 Iteration 4: Addressing Development and Operations Requirements

At some point during the design process, decisions need to be made to satisfy the development and operations requirements. This iteration focuses on decisions made to satisfy QA-7 and CRN-5, which are these types of requirements.

8.4.5.1 Step 2. Establish Iteration Goal by Selecting Drivers

For this iteration, the architect focuses on satisfying the following requirements:

- QA-7 (Deployability): The application is moved between environments as part of the development process. Moving is performed successfully in at most 4 hours.
 - CRN-5: Set up continuous deployment infrastructure.
- Certain constraints are also considered:
- CON-3: Code must be hosted on a proprietary Git-based platform.

8.4.5.2 Step 3. Choose Elements of the System to Refine

For this scenario, the elements that will be refined are the various microservices. However, elements that haven't been identified previously will be added.

8.4.5.3 Step 4. Choose Design Concepts That Satisfy the Selected Drivers

The design concepts used in this iteration are summarized in Table 8.25.

TABLE 8.25 Design Concept Selection Decisions for the Hotel Pricing System: Iteration 4

Design Decision and Location	Rationale
Apply the tactic of managing a deployment pipeline .	The company wants to automate the tasks needed to put the application in the pre-production and production environments. This can be achieved by setting up a CI/CD pipeline.
Apply the tactic of managing the deployed system .	Once built, a container image needs to be uploaded to a container registry so that it can be pulled and instantiated in the different environments. This is necessary for initial deployment and for updates.
Apply the tactic of scripting deployment commands to automate the instantiation of the infrastructure (infrastructure as code).	It is possible to manually set up all the infrastructure in the cloud provider environment, but this is tedious and error-prone. Furthermore, the infrastructures need to be similar in the different environments. Instead of setting them up manually, they are described in a deployment configuration script that can be used to easily instantiate an environment. Also, changes in the configuration of the infrastructure are easily replicated among the different environments.

8.4.5.4 Step 5. Instantiate Architectural Elements, Allocate Responsibilities, and Define Interfaces

The instantiation design decisions are summarized in Table 8.26.

TABLE 8.26 Instantiation Decisions for the Hotel Pricing System: Iteration 4

Design Decision and Location	Rationale
Use proprietary Git-based platform support for CI/CD pipelines .	Constraint CON-3 states that code must be hosted on a proprietary Git-based platform. This platform's support for CI/CD pipelines can be leveraged. Each microservice is managed as an independent project, and each one has to include an individual pipeline. Discarded alternatives included setting up a separate CI server using tools such as Jenkins.
Use cloud provider's container registry to host Docker images.	Because CON-6 states that a cloud-native approach should be favored, it makes sense to leverage the support that the selected cloud provider offers for hosting container images. Discarded alternatives include Docker Hub, because it would require the payment of an additional subscription that is more expensive than the costs of the cloud provider's container registry.
Use cloud provider's support for infrastructure as code to automate instantiation of environments.	Because CON-6 states that a cloud-native approach should be favored, it makes sense to leverage the support that the selected cloud provider offers for describing infrastructure as code. Discarded alternatives include Terraform, because it requires managing external tools; furthermore, the costs incurred by the use of the cloud provider's infrastructure as code features is not significant.

The results of these instantiation decisions are recorded in the next step.

8.4.5.5 Step 6. Sketch Views and Record Design Decisions

Figure 8.15 shows how each microservice is structured as an individual project that is hosted in an individual repository. Table 8.27 describes the elements depicted in the figure.

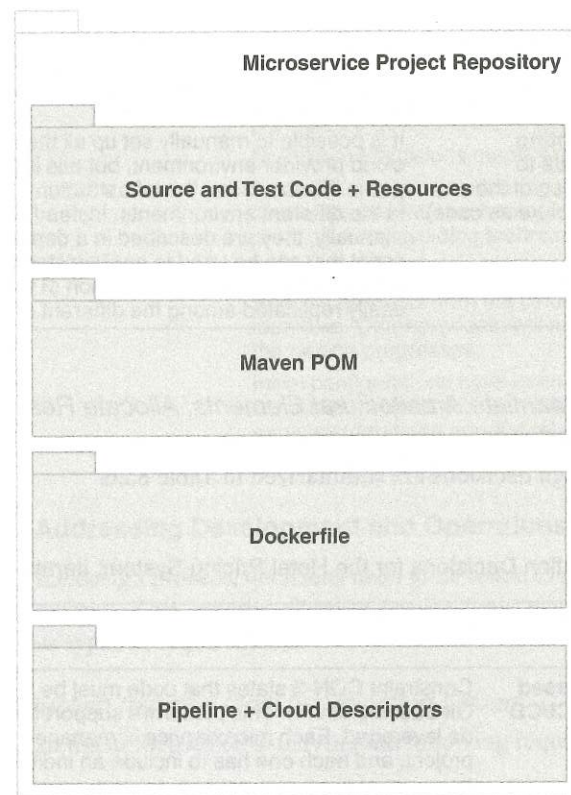


FIGURE 8.15 Package diagram showing structure of repositories (Key: UML)

TABLE 8.27 Repository Elements for the Hotel Pricing System: Iteration 4

Element	Responsibility
Microservice project repository	A project for one microservice, which is hosted in a single repository.
Source and test code + resources	Source and test code, along with resources.
Maven POM	Maven descriptor used to automate the build process.
Dockerfile	Used to build Docker container images.
Pipeline + cloud descriptors	Descriptors that provide information for the CI/CD pipeline steps and for configuring the cloud resources where the microservice is executed.

Figure 8.16 shows the sequence of tasks that triggers the update of a microservice.

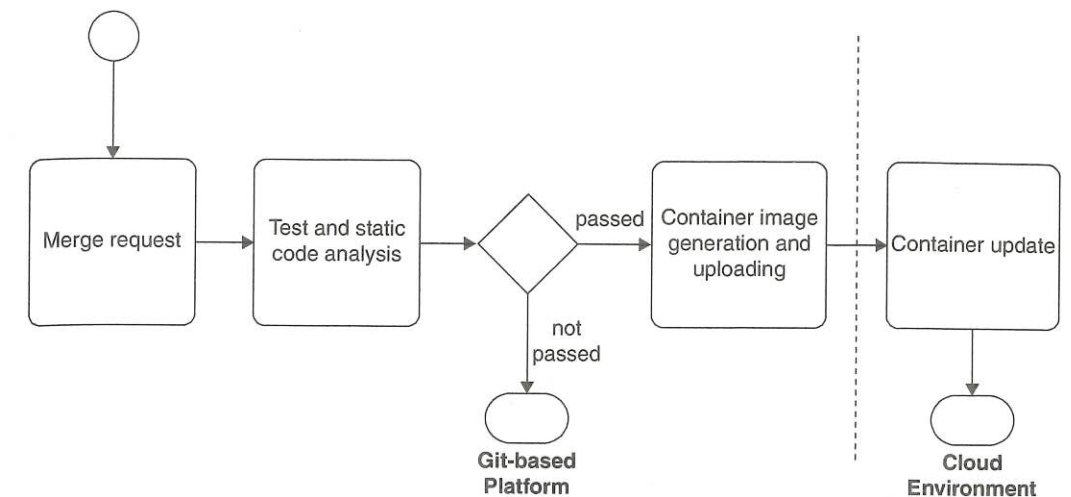


FIGURE 8.16 Activity diagram for the integration and deployment process (Key: UML)

A merge request of a branch with new changes to the main branch triggers the execution of tests and static code analysis (the pipeline descriptor triggers Maven tasks). If the previous step is completed successfully, a container image is generated using the Dockerfile and uploaded to the container registry (this is where the transition from the Git-based platform to the cloud environment occurs). Once uploaded, the container is updated by the orchestrator in the cloud provider's environment.

8.4.5.6 Step 7. Perform Analysis of the Current Design and Review the Iteration Goal and Achievement of Design Purpose

In this iteration, important design decisions have been made to address QA-7 and CRN-5. Table 8.28 summarizes the status of the different drivers and the decisions that were made during the iteration. Note that drivers that were completely addressed in the previous iteration have been removed from the table.

TABLE 8.28 Design Progress for the Hotel Pricing System: Iteration 4

Not Addressed	Partially Addressed	Completely Addressed	Design Decisions Made During the Iteration
	HPS-1		No additional decisions were made.
	HPS-2		No additional decisions were made.
	HPS-4		No additional decisions were made.

continues

Not Addressed	Partially Addressed	Completely Addressed	Design Decisions Made During the Iteration
	HPS-5		No additional decisions were made.
	HPS-6		No additional decisions were made.
	QA-1		No additional decisions were made.
	QA-2		No additional decisions were made.
	QA-3		No additional decisions were made.
	QA-4		No additional decisions were made.
	QA-5		No additional decisions were made.
		QA-7	The use of Docker and externalization of the configuration values plus the establishment of a CI/CD pipeline allows this requirement to be satisfied.
QA-8			No relevant decisions made yet.
		CON-3	The code is now hosted on a Git-based platform.
	CRN-4		No relevant decisions were made during this iteration.
	CRN-5		An infrastructure to support continuous integration and deployment has been established. Further decisions need to be made to manage scenarios such as rollbacks.
	CRN-9		Additional configurations to cloud provider-managed services were made, particularly to the container registry.

At this point, it was determined that sufficient decisions had been made to proceed directly to implementation, without needing further ADD iterations for this initial release of the system. As described in CON-4, the system was slated to have a working MVP within two months, so it was important to transition quickly to an initial implementation.

8.5 Summary

This chapter presented an example of using ADD to design a greenfield system in a mature domain. We first illustrated three iterations with different foci: addressing the general concern of structuring the application, addressing functionality, and addressing quality attribute scenarios. We then illustrated a fourth iteration focused on supporting development and operations requirements.

This example follows the roadmap discussed in Section 4.3.1. It is interesting to note that the application is structured using the CQRS pattern implemented as a set of microservices, but that the microservices themselves are structured internally using a reference architecture. Also, the selection of externally developed components—in this case, frameworks and products, both open source and from a cloud platform—was performed across the different iterations. The example also illustrates how the design results in a system that is functional early in the development process, which aligns with agile development practices. Finally, the example

illustrates that new architectural concerns may appear as the design progresses. CRN-4, the concern associated with avoiding introducing technical debt, is never completely addressed, as the design decisions in each iteration may introduce additional debt.

This example was structured to show how concerns, primary user stories, and quality attribute scenarios can be addressed as part of architectural design. Also, it illustrates that architectural design sometimes requires a certain amount of detail and is more than just a “high level” design. The design was not completed with the iterations described here, and additional iterations would be necessary to satisfy the drivers that were not fully addressed. However, the decisions made within the four iterations were sufficient to demonstrate how ADD is conducted to generate the initial system MVP. Finally, it must be noted that prototypes or analyses need to be created to ensure that the measures associated with the quality attribute scenarios are satisfied.

8.6 Further Reading

Many of the design concepts used in this chapter are discussed in chapters 5, 6 and 7 of this book and also in the online design concepts catalog.

8.7 Discussion Questions

1. A confirmation mechanism for price publication needs to be added to the HPS so that the user interface can show some icons that signal whether a price change is ready to be queried or has been exported to the Channel Management System. Which changes would you make to the architecture to accommodate that requirement?
2. How would you modify the design to support QA-8 (Monitorability)?
3. Several quality attributes were addressed by using cloud-managed solutions. Can all quality attributes be satisfied in this way?
4. Suppose that new hotels are added to the system, and this information needs to be communicated to other systems in the company. Which changes would you make to the architecture to accommodate this requirement?
5. Considering the design decisions that were made during the four iterations, can you think of a different ordering for them? For example, can the deployment and operations design decisions be made prior to the ones made in the earlier iterations?
6. Consider CON-5: The system must interact initially with existing systems through REST APIs but may need to later support other protocols. Which changes would you make to the design of the system, if any, to accommodate this constraint?